Bikkad IT

Dream Big! Start Small!!! Act Now!!!

Bikkad IT Institute

Session 2: Operators, If-Else & Loops

Practice Questions with Python Code Answers

Compiled by Bikkad IT Institute | Teaching Notes

WHAT IS INSIDE

1. Arithmetic Operators
2. Relational Operators
3. Logical Operators
4. Bitwise Operators
5. Assignment Operators
6. Membership Operators
7. If-Else Statements
8. Math Module
9. Random Module
10. Datetime Module
11. While Loop
12. For Loop
13. Nested Loops

78 practice questions | every topic: 2 Beginner + 2 Intermediate + 2 Advanced

BEGINNER INTERMEDIATE ADVANCED

OUR LOCATIONS

+91 77220 82225 Pimpri-Chinchwad

+91 77220 82226 Chh. Sambhajinagar

+91 77220 82227 Nashik

+91 77220 82224 Hadapsar, Pune

Session 2: Operators, If-Else & Loops - Practice Solutions

Prepared for: Bikkad IT Institute Students
Topics: Arithmetic Operators, Relational Operators, Logical Operators, Bitwise Operators, Assignment Operators, Membership Operators, If-Else Statements, Math Module, Random Module, Datetime Module, While Loop, For Loop, Nested Loops
Levels: 2 Beginner + 2 Intermediate + 2 Advanced questions per topic
Date: October 2026

1. Arithmetic Operators

Q1. Split the Restaurant Bill

BEGINNER

Five friends had dinner and the bill is Rs. 2350. Calculate how much each friend has to pay.

Solution:

```
bill = 2350
friends = 5

per_person = bill / friends
print(per_person)
```

Output:

```
470.0
```

Q2. Farm Area and Fencing

BEGINNER

A rectangular farm is 120 m long and 80 m wide. Calculate its area (length x breadth) and the fencing required (perimeter = 2 x (length + breadth)).

Solution:

```
length = 120
breadth = 80

area = length * breadth
print(area)

perimeter = 2 * (length + breadth)
print(perimeter)
```

Output:

```
9600
400
```

Q3. Video Duration Converter

INTERMEDIATE

A YouTube video is 1000 seconds long. Convert it into minutes and remaining seconds using // and %.

Solution:

```
total_seconds = 1000

minutes = total_seconds // 60
print(minutes)

seconds = total_seconds % 60
print(seconds)
```

Output:

```
16
40
```

Q4. Packing Eggs in Trays

INTERMEDIATE

A poultry farm collected 250 eggs. One tray holds 12 eggs. Find how many full trays can be packed and how many eggs remain.

Solution:

```
eggs = 250
tray_size = 12

full_trays = eggs // tray_size
print(full_trays)

remaining = eggs % tray_size
print(remaining)
```

Output:

```
20
10
```

Q5. Fixed Deposit Maturity

ADVANCED

A fixed deposit of Rs. 50000 earns 7% interest compounded yearly for 5 years. Calculate the maturity amount using A = P x (1 + R/100) ** T and the interest earned. Round to 2 decimals.

Solution:

```
principal = 50000
rate = 7
years = 5

amount = principal * (1 + rate / 100) ** years
print(round(amount, 2))

interest = amount - principal
print(round(interest, 2))
```

Output:

```
70127.59
20127.59
```

Q6. Reverse a 3-Digit Number

ADVANCED

Without using strings or loops, reverse the 3-digit bus number 472 using only // and %. Also find the sum of its digits.

Solution:

```
number = 472

hundreds = number // 100
tens = (number // 10) % 10
units = number % 10

reversed_number = units * 100 + tens * 10 + hundreds
print(reversed_number)

digit_sum = hundreds + tens + units
print(digit_sum)
```

Output:

```
274
13
```

2. Relational Operators

Q7. Enough Balance to Withdraw?

BEGINNER

An ATM account has balance 7500. The customer wants to withdraw 8000. Use a relational operator to check whether the balance is enough and print the result.

Solution:

```
balance = 7500
withdraw = 8000

can_withdraw = balance >= withdraw
print(can_withdraw)
```

Output:

```
False
```

Q8. Compare Product Prices

BEGINNER

The same headphone costs 1499 on Shop A and 1499 on Shop B. Check if the prices are equal and if they are not equal. Print both results.

Solution:

```
shop_a = 1499
shop_b = 1499

print(shop_a == shop_b)
print(shop_a != shop_b)
```

Output:

```
True
False
```

Q9. Voting Eligibility

INTERMEDIATE

Store the ages of three citizens (17, 18, 45). For each, check whether they are eligible to vote (age >= 18) and print True/False.

Solution:

```
age1 = 17
age2 = 18
age3 = 45

print(age1 >= 18)
print(age2 >= 18)
print(age3 >= 18)
```

Output:

```
False
True
True
```

Q10. Chained Comparison

INTERMEDIATE

A company hires people aged between 21 and 58 (both included). Using a chained comparison (21 <= age <= 58), check the ages 20, 35 and 58.

Solution:

```
age1 = 20
age2 = 35
age3 = 58

print(21 <= age1 <= 58)
print(21 <= age2 <= 58)
print(21 <= age3 <= 58)
```

Output:

```
False
True
True
```

Q11. Comparing Text Values

ADVANCED

Relational operators also work on strings (dictionary order). Compare "apple" < "mango", "Zebra" < "apple" and "pune" == "Pune". Use ord() to explain why "Zebra" comes before "apple".

Solution:

```
print("apple" < "mango")
print("Zebra" < "apple")
print("pune" == "Pune")

print(ord("Z"))
print(ord("a"))
# Capital letters have smaller codes (A-Z = 65-90, a-z = 97-122)
```

Output:

```
True
True
False
90
97
```

Q12. Floating Point Comparison Trap

ADVANCED

A payment app adds 0.1 and 0.2 rupees. Check whether the result == 0.3. You will get False. Fix the comparison using round() and also using abs(difference) < 0.000001.

Solution:

```
total = 0.1 + 0.2
print(total)
print(total == 0.3)

print(round(total, 2) == 0.3)

difference = abs(total - 0.3)
print(difference < 0.000001)
```

Output:

```
0.30000000000000004
False
True
True
```

3. Logical Operators

Q13. Simple Login Check

BEGINNER

The correct username is "admin" and password is "Bikkad@123". The user entered "admin" and "Bikkad@123". Use the and operator to check if login is successful.

Solution:

```
correct_user = "admin"
correct_pass = "Bikkad@123"

entered_user = "admin"
entered_pass = "Bikkad@123"

login_ok = (entered_user == correct_user
            and entered_pass == correct_pass)
print(login_ok)
```

Output:

```
True
```

Q14. Weekend Discount

BEGINNER

A cafe gives a discount on Saturday or Sunday. Store today as "Sunday" and use the or operator to check if the discount applies.

Solution:

```
day = "Sunday"

discount = day == "Saturday" or day == "Sunday"
print(discount)
```

Output:

```
True
```

Q15. Loan Eligibility

INTERMEDIATE

A bank gives a loan if salary is at least 25000 AND credit score is at least 700 AND the customer has NOT defaulted earlier. Check for salary 32000, credit score 720, has_defaulted False.

Solution:

```
salary = 32000
credit_score = 720
has_defaulted = False

eligible = (salary >= 25000
            and credit_score >= 700
            and not has_defaulted)
print(eligible)
```

Output:

```
True
```

Q16. Store Open or Closed

INTERMEDIATE

A shop is open from 9 to 21 (24-hour clock). For the current hour 22, find is_open and then use not to find is_closed.

Solution:

```
hour = 22

is_open = hour >= 9 and hour < 21
print(is_open)

is_closed = not is_open
print(is_closed)
```

Output:

```
False
True
```

Q17. Default Value with or

ADVANCED

If a user leaves the name field empty, the app should show "Guest". Use the short-circuit behaviour of or: display_name = user_name or "Guest". Test with an empty name and with "Nikhil".

Solution:

```
user_name = ""
display_name = user_name or "Guest"
print(display_name)

user_name = "Nikhil"
display_name = user_name or "Guest"
print(display_name)
```

Output:

```
Guest
Nikhil
```

Q18. Leap Year Rule

ADVANCED

A year is a leap year if it is divisible by 4 and not by 100, OR it is divisible by 400. Write one logical expression and test it for 2024, 1900, 2000 and 2026.

Solution:

```
year = 2024
print((year % 4 == 0 and year % 100 != 0) or year % 400 == 0)

year = 1900
print((year % 4 == 0 and year % 100 != 0) or year % 400 == 0)

year = 2000
print((year % 4 == 0 and year % 100 != 0) or year % 400 == 0)

year = 2026
print((year % 4 == 0 and year % 100 != 0) or year % 400 == 0)
```

Output:

```
True
False
True
False
```

4. Bitwise Operators

Q19. AND, OR, XOR of Two Numbers

BEGINNER

Take a = 12 and b = 10. Print their binary form using bin() and the results of a & b, a | b and a ^ b.

Solution:

```
a = 12
b = 10

print(bin(a))
print(bin(b))
print(a & b)
print(a | b)
print(a ^ b)
```

Output:

```
0b1100
0b1010
8
14
6
```

Q20. Even or Odd Using Bitwise AND

BEGINNER

The last binary bit of an odd number is always 1. Use n & 1 to check whether the token numbers 47 and 82 are odd (result 1) or even (result 0).

Solution:

```
token1 = 47
token2 = 82

print(token1 & 1)
print(token2 & 1)
```

Output:

```
1
0
```

Q21. Fast Multiply and Divide

INTERMEDIATE

Left shift by 1 multiplies by 2, right shift by 1 divides by 2. Use shifts to calculate 10 x 2, 10 x 8 and 100 / 4 (integer).

Solution:

```
num = 10
print(num << 1)
print(num << 3)

big = 100
print(big >> 2)
```

Output:

```
20
80
25
```

Q22. Swap Using XOR

INTERMEDIATE

Swap two locker numbers a = 25 and b = 40 without using a third variable, using the XOR (^) operator three times.

Solution:

```
a = 25
b = 40

a = a ^ b
b = a ^ b
a = a ^ b

print(a)
print(b)
```

Output:

```
40
25
```

Q23. File Permission Flags

ADVANCED

Operating systems store permissions as bits: READ = 4, WRITE = 2, EXECUTE = 1. Give a user READ and WRITE permission using |. Then check if the user has WRITE and EXECUTE permission using &.

Solution:

```
READ = 4
WRITE = 2
EXECUTE = 1

user_perm = READ | WRITE
print(user_perm)

print(user_perm & WRITE != 0)
print(user_perm & EXECUTE != 0)
```

Output:

```
6
True
False
```

Q24. Bit Tricks

ADVANCED

For number 40: (1) count how many bits are 1 using bin().count("1"), (2) toggle the bit at position 3 using ^ (1 << 3), (3) check if 64 and 40 are powers of two using n & (n - 1) == 0.

Solution:

```
n = 40
print(bin(n))
print(bin(n).count("1"))

toggled = n ^ (1 << 3)
print(toggled)

print(64 & (64 - 1) == 0)
print(40 & (40 - 1) == 0)
```

Output:

```
0b101000
2
32
True
False
```

5. Assignment Operators

Q25. Piggy Bank Savings

BEGINNER

A child has Rs. 100 in a piggy bank and adds Rs. 50 every week for 3 weeks. Use += to update the savings and print it after each week.

Solution:

```
savings = 100

savings += 50
print(savings)

savings += 50
print(savings)

savings += 50
print(savings)
```

Output:

```
150
200
250
```

Q26. Warehouse Stock

BEGINNER

A warehouse has 500 bags of rice. On Monday 120 are sold and on Tuesday 95 are sold. Update the stock using -= and print the final stock.

Solution:

```
stock = 500

stock -= 120
print(stock)

stock -= 95
print(stock)
```

Output:

```
380
285
```

Q27. Investment Doubling

INTERMEDIATE

An investment of 10000 doubles every 6 years. Use *= to find its value after 6, 12 and 18 years.

Solution:

```
investment = 10000

investment *= 2
print(investment)

investment *= 2
print(investment)

investment *= 2
print(investment)
```

Output:

```
20000
40000
80000
```

Q28. Pizza Slices

INTERMEDIATE

A party has 37 pizza slices for 8 people. Store slices in a variable and use //= to find slices per person; in another variable use %= to find leftover slices.

Solution:

```
slices = 37
per_person = slices
per_person //= 8
print(per_person)

leftover = slices
leftover %= 8
print(leftover)
```

Output:

```
4
5
```

Q29. Price Changes in Sequence

ADVANCED

A TV costs 40000. Apply in sequence: 12% festival discount (*= 0.88), then /= 2 for a two-month EMI, then %= 1000 to find the amount above the last full thousand. Finally find the area of a square tile of side 3 using **=.

Solution:

```
price = 40000

price *= 0.88
print(price)

price /= 2
print(price)

price %= 1000
print(price)

tile = 3
tile **= 2
print(tile)
```

Output:

```
35200.0
17600.0
600.0
9
```

Q30. Walrus Operator :=

ADVANCED

Python 3.8 added the walrus operator := which assigns and returns a value in one step. For a password "Pune@2026", assign its length with := inside an if condition and print a message if the length is at least 8.

Solution:

```
password = "Pune@2026"

if (length := len(password)) >= 8:
    print("Strong length:", length)
else:
    print("Too short:", length)
```

Output:

```
Strong length: 9
```

6. Membership Operators

Q31. Serviceable City

BEGINNER

A food delivery app serves ["Pune", "Mumbai", "Nashik", "Nagpur"]. Check whether "Nashik" and "Satara" are serviceable.

Solution:

```
cities = ["Pune", "Mumbai", "Nashik", "Nagpur"]

print("Nashik" in cities)
print("Satara" in cities)
```

Output:

```
True
False
```

Q32. Basic Email Check

BEGINNER

Check whether the email "student.bikkad@gmail.com" contains the "@" symbol and does NOT contain a space.

Solution:

```
email = "student.bikkad@gmail.com"

print("@" in email)
print(" " not in email)
```

Output:

```
True
True
```

Q33. Spam Word Filter

INTERMEDIATE

A comment says "Click here to win FREE money". Check whether the words "FREE" and "lottery" appear in it.

Solution:

```
comment = "Click here to win FREE money"

print("FREE" in comment)
print("lottery" in comment)
```

Output:

```
True
False
```

Q34. Attendance Register

INTERMEDIATE

Attendance is stored as a dictionary {101: "Present", 102: "Absent", 103: "Present"}. Check if roll numbers 102 and 110 exist. (Note: in checks the KEYS of a dictionary.)

Solution:

```
attendance = {101: "Present", 102: "Absent", 103: "Present"}

print(102 in attendance)
print(110 in attendance)
print("Absent" in attendance)
```

Output:

```
True
False
False
```

Q35. Delivery PIN Code Rules

ADVANCED

Deliverable PIN codes are (411001, 411014, 411038) and blocked PIN codes are {411014}. For PIN 411014, check that it is in the deliverable tuple AND not in the blocked set, and print whether delivery is possible.

Solution:

```
deliverable = (411001, 411014, 411038)
blocked = {411014}

pin = 411014
can_deliver = pin in deliverable and pin not in blocked

if can_deliver:
    print("Delivery available")
else:
    print("Delivery not available")
```

Output:

```
Delivery not available
```

Q36. Membership with range

ADVANCED

Check whether ages 25 and 65 are in the working age range(18, 61). Then check whether the number 7 is among the odd numbers below 20 using range(1, 20, 2).

Solution:

```
print(25 in range(18, 61))
print(65 in range(18, 61))

print(7 in range(1, 20, 2))
print(8 in range(1, 20, 2))
```

Output:

```
True
False
True
False
```

7. If-Else Statements

Q37. Odd or Even Token

BEGINNER

A bank token number is 57. Print "Even" if it is even, otherwise print "Odd".

Solution:

```
token = 57

if token % 2 == 0:
    print("Even")
else:
    print("Odd")
```

Output:

```
Odd
```

Q38. Pass or Fail

BEGINNER

A student scored 32 marks. Passing marks are 35. Print "Pass" or "Fail".

Solution:

```
marks = 32

if marks >= 35:
    print("Pass")
else:
    print("Fail")
```

Output:

```
Fail
```

Q39. Electricity Bill Slabs

INTERMEDIATE

Electricity charges: first 100 units Rs. 5/unit, next 100 units (101-200) Rs. 7/unit, above 200 units Rs. 10/unit. Calculate the bill for 260 units using if-elif-else.

Solution:

```
units = 260

if units <= 100:
    bill = units * 5
elif units <= 200:
    bill = 100 * 5 + (units - 100) * 7
else:
    bill = 100 * 5 + 100 * 7 + (units - 200) * 10

print(bill)
```

Output:

```
1800
```

Q40. Highest Scorer

INTERMEDIATE

Three players scored 45, 78 and 63 runs. Find the highest score using if-elif-else (without using max()).

Solution:

```
a = 45
b = 78
c = 63

if a >= b and a >= c:
    highest = a
elif b >= a and b >= c:
    highest = b
else:
    highest = c

print(highest)
```

Output:

```
78
```

Q41. Income Tax Calculator (Practice Slabs)

ADVANCED

Use these practice slabs: up to 3,00,000 = 0%; 3,00,001 to 7,00,000 = 5% on amount above 3L; 7,00,001 to 10,00,000 = 20000 + 10% above 7L; above 10L = 50000 + 20% above 10L. Find tax for income 12,50,000.

Solution:

```
income = 1250000

if income <= 300000:
    tax = 0
elif income <= 700000:
    tax = (income - 300000) * 5 / 100
elif income <= 1000000:
    tax = 20000 + (income - 700000) * 10 / 100
else:
    tax = 50000 + (income - 1000000) * 20 / 100

print(tax)
```

Output:

```
100000.0
```

Q42. ATM Withdrawal (Nested If)

ADVANCED

An ATM allows withdrawal only if: PIN is correct (1234), amount is a multiple of 100, and amount is not more than balance (15000). Use nested if to print a suitable message for PIN 1234 and amount 4500.

Solution:

```
correct_pin = 1234
balance = 15000

pin = 1234
amount = 4500

if pin == correct_pin:
    if amount % 100 == 0:
        if amount <= balance:
            balance = balance - amount
            print("Please collect cash")
            print(balance)
        else:
            print("Insufficient balance")
    else:
        print("Enter amount in multiples of 100")
else:
    print("Wrong PIN")
```

Output:

```
Please collect cash
10500
```

8. Math Module

Q43. Side of a Square Plot

BEGINNER

A square plot has an area of 2025 sq. ft. Find the length of one side using math.sqrt().

Solution:

```
import math

area = 2025
side = math.sqrt(area)
print(side)
```

Output:

```
45.0
```

Q44. Buses for a School Trip

BEGINNER

157 students are going on a trip and each bus holds 40 students. Use math.ceil() to find the number of buses needed.

Solution:

```
import math

students = 157
capacity = 40

buses = math.ceil(students / capacity)
print(buses)
```

Output:

```
4
```

Q45. Which Pizza Gives More?

INTERMEDIATE

Is one large pizza (radius 15 cm) bigger than two small pizzas (radius 9 cm each)? Calculate areas using math.pi and compare.

Solution:

```
import math

large = math.pi * 15 ** 2
print(round(large, 2))

two_small = 2 * math.pi * 9 ** 2
print(round(two_small, 2))

print(large > two_small)
```

Output:

```
706.86
508.94
True
```

Q46. Arranging Books on a Shelf

INTERMEDIATE

In how many different ways can 6 different books be arranged on a shelf? (Use math.factorial). Also round down 7.9 and round up 7.1 using floor and ceil.

Solution:

```
import math

ways = math.factorial(6)
print(ways)

print(math.floor(7.9))
print(math.ceil(7.1))
```

Output:

```
720
7
8
```

Q47. Ladder Against a Wall

ADVANCED

A ladder's foot is 6 m away from a wall and it reaches 8 m high on the wall. Find the ladder length using math.sqrt(a**2 + b**2) and also using math.hypot(). Find the distance between two shops at points (2, 3) and (7, 15) using math.dist().

Solution:

```
import math

base = 6
height = 8

ladder = math.sqrt(base ** 2 + height ** 2)
print(ladder)

ladder2 = math.hypot(base, height)
print(ladder2)

shop_a = (2, 3)
shop_b = (7, 15)
distance = math.dist(shop_a, shop_b)
print(distance)
```

Output:

```
10.0
10.0
13.0
```

Q48. Bus Timings and Lottery Combinations

ADVANCED

Bus A leaves every 12 minutes and Bus B every 18 minutes. Using math.lcm() find after how many minutes both leave together; using math.gcd() find the largest equal time slot. Also find how many ways 6 numbers can be chosen from 49 using math.comb().

Solution:

```
import math

bus_a = 12
bus_b = 18

together = math.lcm(bus_a, bus_b)
print(together)

slot = math.gcd(bus_a, bus_b)
print(slot)

combinations = math.comb(49, 6)
print(combinations)
```

Output:

```
36
6
13983816
```

9. Random Module

Q49. Ludo Dice Roll

BEGINNER

Simulate rolling a dice for a Ludo game using random.randint(1, 6). Roll it two times and print both values.

Solution:

```
import random

roll1 = random.randint(1, 6)
print(roll1)

roll2 = random.randint(1, 6)
print(roll2)
```

Sample Output (changes on every run):

```
4
5
```

Q50. Lucky Draw Winner

BEGINNER

A shop runs a lucky draw for customers ["Anita", "Sagar", "Farhan", "Meena", "Joseph"]. Pick one winner using random.choice().

Solution:

```
import random

customers = ["Anita", "Sagar", "Farhan", "Meena", "Joseph"]
winner = random.choice(customers)
print(winner)
```

Sample Output (changes on every run):

```
Meena
```

Q51. 6-Digit OTP

INTERMEDIATE

Generate a 6-digit OTP for a banking app (between 100000 and 999999) and a random float between 0 and 1 using random.random().

Solution:

```
import random

otp = random.randint(100000, 999999)
print(otp)

value = random.random()
print(value)
```

Sample Output (changes on every run):

```
574354
0.8657422852499215
```

Q52. Shuffle Quiz Questions

INTERMEDIATE

To prevent copying, a teacher shuffles the question numbers [1, 2, 3, 4, 5, 6, 7, 8] for every student. Use random.shuffle() and print the order.

Solution:

```
import random

questions = [1, 2, 3, 4, 5, 6, 7, 8]
random.shuffle(questions)
print(questions)
```

Sample Output (changes on every run):

```
[2, 1, 3, 6, 4, 5, 7, 8]
```

Q53. Random Password Generator

ADVANCED

Create an 8-character password using a for loop and random.choice() from the characters "ABCDEFGHJKLMNPQRSTUVWXYZabcdefghijkmnpqrstuvwxyz23456789@#$".

Solution:

```
import random

upper = "ABCDEFGHJKLMNPQRSTUVWXYZ"
lower = "abcdefghijkmnpqrstuvwxyz"
others = "23456789@#$"
characters = upper + lower + others
password = ""

for i in range(8):
    password = password + random.choice(characters)

print(password)
```

Sample Output (changes on every run):

```
e9m83fei
```

Q54. Coin Toss Simulation with seed

ADVANCED

Use random.seed(42) so that results are repeatable. Toss a coin 1000 times using random.choice(["H", "T"]) and count heads and tails. Print both counts.

Solution:

```
import random

random.seed(42)
heads = 0
tails = 0

for i in range(1000):
    toss = random.choice(["H", "T"])
    if toss == "H":
        heads += 1
    else:
        tails += 1

print(heads)
print(tails)
```

Output:

```
486
514
```

10. Datetime Module

Q55. Today's Date and Time

BEGINNER

Print the current date and time using datetime.datetime.now() and only today's date using datetime.date.today().

Solution:

```
import datetime

now = datetime.datetime.now()
print(now)

today = datetime.date.today()
print(today)
```

Sample Output (changes on every run):

```
2026-10-17 06:21:54.883674
2026-10-17
```

Q56. Parts of an Exam Date

BEGINNER

The exam date is 15 March 2027. Create it using datetime.date(2027, 3, 15) and print the year, month and day separately.

Solution:

```
import datetime

exam = datetime.date(2027, 3, 15)

print(exam.year)
print(exam.month)
print(exam.day)
```

Output:

```
2027
3
15
```

Q57. Format a Bill Date

INTERMEDIATE

A bill was generated on datetime(2026, 10, 17, 14, 30). Print it in the formats "17-10-2026", "17 October 2026", "Saturday" and "02:30 PM" using strftime().

Solution:

```
import datetime

bill_time = datetime.datetime(2026, 10, 17, 14, 30)

print(bill_time.strftime("%d-%m-%Y"))
print(bill_time.strftime("%d %B %Y"))
print(bill_time.strftime("%A"))
print(bill_time.strftime("%I:%M %p"))
```

Output:

```
17-10-2026
17 October 2026
Saturday
02:30 PM
```

Q58. Age from Birth Year

INTERMEDIATE

A student was born in 2004. Calculate the age using the year of a fixed date datetime.date(2026, 10, 17).

Solution:

```
import datetime

birth_year = 2004
today = datetime.date(2026, 10, 17)

age = today.year - birth_year
print(age)
```

Output:

```
22
```

Q59. Days Left for Project Deadline

ADVANCED

A project started on 17 Oct 2026 and the deadline is 31 Dec 2026. Subtract the dates to find days left, and also find how many complete weeks that is.

Solution:

```
import datetime

start = datetime.date(2026, 10, 17)
deadline = datetime.date(2026, 12, 31)

gap = deadline - start
print(gap)
print(gap.days)

weeks = gap.days // 7
print(weeks)
```

Output:

```
75 days, 0:00:00
75
10
```

Q60. Library Due Date and Fine

ADVANCED

A book is issued on 1 Oct 2026 for 14 days (use timedelta). It is returned on 22 Oct 2026. Find the due date, late days, and fine at Rs. 5 per late day.

Solution:

```
import datetime

issue_date = datetime.date(2026, 10, 1)
due_date = issue_date + datetime.timedelta(days=14)
print(due_date)

return_date = datetime.date(2026, 10, 22)
late_days = (return_date - due_date).days
print(late_days)

if late_days > 0:
    fine = late_days * 5
else:
    fine = 0
print(fine)
```

Output:

```
2026-10-15
7
35
```

11. While Loop

Q61. Rocket Launch Countdown

BEGINNER

Print a countdown from 5 to 1 and then print "Launch!" using a while loop.

Solution:

```
count = 5

while count >= 1:
    print(count)
    count -= 1

print("Launch!")
```

Output:

```
5
4
3
2
1
Launch!
```

Q62. Sum of First 10 Numbers

BEGINNER

Using a while loop, find the sum of numbers from 1 to 10.

Solution:

```
i = 1
total = 0

while i <= 10:
    total += i
    i += 1

print(total)
```

Output:

```
55
```

Q63. ATM PIN with 3 Attempts (while-else)

INTERMEDIATE

Allow the user 3 attempts to enter the correct PIN (4321). If correct, print "Access granted" and stop. If all 3 attempts fail, the else part of the while loop should print "Card blocked".

Solution:

```
correct_pin = 4321
attempts = 1

while attempts <= 3:
    pin = int(input("Enter PIN: "))
    if pin == correct_pin:
        print("Access granted")
        break
    print("Wrong PIN")
    attempts += 1
else:
    print("Card blocked")
```

Output (with sample input):

```
Enter PIN: 1111
Wrong PIN
Enter PIN: 4321
Access granted
```

Q64. Sum of Digits

INTERMEDIATE

Find the sum of digits of the vehicle number 8456 using a while loop with % and //.

Solution:

```
number = 8456
digit_sum = 0

while number > 0:
    digit = number % 10
    digit_sum += digit
    number = number // 10

print(digit_sum)
```

Output:

```
23
```

Q65. Palindrome Number

ADVANCED

Check whether the number 12321 is a palindrome (same when reversed) by reversing it with a while loop.

Solution:

```
number = 12321
original = number
reverse = 0

while number > 0:
    digit = number % 10
    reverse = reverse * 10 + digit
    number = number // 10

print(reverse)

if reverse == original:
    print("Palindrome")
else:
    print("Not palindrome")
```

Output:

```
12321
Palindrome
```

Q66. Months to Reach Savings Goal

ADVANCED

Riya saves Rs. 3000 every month, and her total savings earn 1% interest every month. Use a while loop to find how many months she needs to reach Rs. 50000.

Solution:

```
savings = 0
months = 0
target = 50000

while savings < target:
    savings = savings + 3000
    savings = savings + savings * 1 / 100
    months += 1

print(months)
print(round(savings, 2))
```

Output:

```
16
52291.33
```

12. For Loop

Q67. Shopping List

BEGINNER

Print every item of the shopping list ["Milk", "Bread", "Eggs", "Rice"] using a for loop.

Solution:

```
shopping = ["Milk", "Bread", "Eggs", "Rice"]

for item in shopping:
    print(item)
```

Output:

```
Milk
Bread
Eggs
Rice
```

Q68. Table of 9

BEGINNER

Print the multiplication table of 9 from 1 to 10 using a for loop and range().

Solution:

```
num = 9

for i in range(1, 11):
    print(num, "x", i, "=", num * i)
```

Output:

```
9 x 1 = 9
9 x 2 = 18
9 x 3 = 27
9 x 4 = 36
9 x 5 = 45
9 x 6 = 54
9 x 7 = 63
9 x 8 = 72
9 x 9 = 81
9 x 10 = 90
```

Q69. Sum of Even Numbers

INTERMEDIATE

Find the sum of all even numbers from 1 to 100 using range() with a step.

Solution:

```
total = 0

for n in range(2, 101, 2):
    total += n

print(total)
```

Output:

```
2550
```

Q70. Count Vowels in a City Name

INTERMEDIATE

Count the number of vowels in the word "Aurangabad" using a for loop and the in operator.

Solution:

```
city = "Aurangabad"
count = 0

for ch in city:
    if ch in "aeiouAEIOU":
        count += 1

print(count)
```

Output:

```
5
```

Q71. Factorial for Seating Arrangements

ADVANCED

Calculate 7! (the number of ways 7 guests can sit in a row) using a for loop.

Solution:

```
n = 7
fact = 1

for i in range(1, n + 1):
    fact = fact * i

print(fact)
```

Output:

```
5040
```

Q72. Prime Numbers with for-else

ADVANCED

Print all prime numbers between 2 and 30. Use a nested for loop with break and the for-else feature.

Solution:

```
for num in range(2, 31):
    for d in range(2, num):
        if num % d == 0:
            break
    else:
        print(num, end=" ")
print()
```

Output:

```
2 3 5 7 11 13 17 19 23 29
```

13. Nested Loops

Q73. Square Star Pattern

BEGINNER

Print a 4 x 4 square of stars using nested for loops.

Solution:

```
for row in range(4):
    for col in range(4):
        print("*", end=" ")
    print()
```

Output:

```
* * * *
* * * *
* * * *
* * * *
```

Q74. Theatre Seat Numbers

BEGINNER

A small theatre has rows A, B, C and 4 seats in each row. Print all seat numbers like A1 A2 A3 A4 row by row.

Solution:

```
for row in "ABC":
    for seat in range(1, 5):
        print(row + str(seat), end=" ")
    print()
```

Output:

```
A1 A2 A3 A4
B1 B2 B3 B4
C1 C2 C3 C4
```

Q75. Repeated Number Pattern

INTERMEDIATE

Print the pattern:
1
2 2
3 3 3
4 4 4 4

Solution:

```
for i in range(1, 5):
    for j in range(i):
        print(i, end=" ")
    print()
```

Output:

```
1
2 2
3 3 3
4 4 4 4
```

Q76. Multiplication Grid

INTERMEDIATE

Print a 5 x 5 multiplication grid (row x column) using nested loops. Use \t to align the columns.

Solution:

```
for i in range(1, 6):
    for j in range(1, 6):
        print(i * j, end="\t")
    print()
```

Output:

```
1       2       3       4       5
2       4       6       8       10
3       6       9       12      15
4       8       12      16      20
5       10      15      20      25
```

Q77. Pyramid Pattern

ADVANCED

Print a centered pyramid of stars with 5 rows using nested loops (spaces first, then stars).

Solution:

```
rows = 5

for i in range(1, rows + 1):
    for s in range(rows - i):
        print(" ", end="")
    for k in range(2 * i - 1):
        print("*", end="")
    print()
```

Output:

```
    *
   ***
  *****
 *******
*********
```

Q78. Gift Combos Within Budget

ADVANCED

A shop sells shirts [499, 799, 999] and jeans [999, 1299, 1499]. Using nested loops, print every shirt + jeans combination whose total is less than the budget of Rs. 1800.

Solution:

```
shirts = [499, 799, 999]
jeans = [999, 1299, 1499]
budget = 1800

for s in shirts:
    for j in jeans:
        if s + j < budget:
            print(s, j, s + j)
```

Output:

```
499 999 1498
499 1299 1798
799 999 1798
```

End of Solutions

Prepared by: Bikkad IT Institute
For: Python Beginner Level Students
Date: October 2026